Lab 2: Single-Cycle RISC-V Microarchitecture Implementation in SystemVerilog HDL

04/3/2026

1st Thomas Parsley
School of Electrical and Computer Engineering
Oklahoma State University
Stillwater, Oklahoma
thomas.parsley@okstate.edu
A20431711

2nd Nick Platon
School of Electrical and Computer Engineering
Oklahoma State University
Stillwater, Oklahoma
nplaton@okstate.edu
A20413415

I. Introduction

In Lab 2, we designed, simulated, and implemented RISC-V 32 bit instructions. In SystemVerilog, we added commonly used instructions in RISC-V 32I architecture, with at least one instruction for I-type, S-type, U-type, B-type, R-type and J-type instructions. We simulated each instruction of the instruction set, verifying that the instructions work properly through Assembly programs, which were converted to machine code, that are specifically designed to validate that instruction. Once all instructions had been validated, we then implemented the design on the FPGA/DSDB boards using Vivado. This lab builds on knowledge from previous labs, such as understanding the functionality of a register file module like the one we designed in Lab 0, to design the processor. It also aligns with current lecture topics, as we have been covering single cycle processors and the datapath design during lecture.

II. Design Overview

Our design requirement for this lab was to create additional instructions for the unfinished RISC-V single cycle processor given in the lab documents. There was at least one instruction that was required to be designed from each of the six instruction types. Figure 2.1 shows a table showing the required instructions for this design, with the crossed out instructions being previously made and given to us in the lab documents.

~~add~~	~~addi~~	~~andi~~	~~and~~	auipc
~~beq~~	bge	bgeu	blt	bltu
bne	jalr	~~jal~~	lb	lbu
lh	lhu	lui	~~lw~~	~~or~~
~~ori~~	sb	sh	sll	slli
~~slt~~	~~slti~~	sltiu	sltu	sra
srai	srl	srli	~~sub~~	~~sw~~
xor	xori			

[1](Figure 2.1 - Required Instructions for Lab 2)

For the R-type instructions, we edited the ALU to include all operations. This required us to change the ALU Decoder to use func3 to separate each instruction into an alucontrol value. Then the ALU would take the ALUControl value and use a case statement to decide the result, whether that be the sum, a comparison, or a shift. This result is put into a mux that chooses what to write into the register file.

For the I-type instructions, we had many changes to the datapath since there are vastly different functions that are all I-type. The arithmetic I-type instructions would follow a similar path through the ALU and register file like the R-type instructions, however the B source for the ALU would be substituted with an immediate, which would already be extended. The decision for either a read register from the register file and an immediate from the immediate extension was controlled by a mux that uses the ALUSrc,

which was defined by the main decoder from the OP codes. Later in the design, the immediate extension module had to be edited to account for other instruction types that utilize immediates. The load instructions required us to take the read data coming from the memory data and decide which parts of the data were supposed to be loaded. For this, a mux for loading bytes and a mux for loading half words were created. These muxes would take the read data from the memory and chose which part of it needed to be loaded (i.e. Bytes could be [7:0], [15:8], [23:16], or [31:24]). Once both a byte and halfword were selected, they would go into another mux that decided between full word, signed byte, unsigned byte, signed half word, and unsigned half word, with the decision value being Func3. For the unsigned values, the byte or half word would get zero extended, and for the signed values, the byte or half word would get sign extended. The full word remained the same as the read data from the memory. The result of this would be put into the mux that decides the write data to the register file mentioned earlier.

For the U-type instructions, we only had two instructions to implement. We took the immediate from the instruction and extended it in the immediate extend module, where 12'b0 were added after the least significant bit. For LUI, we used the ALU, only setting the result of the ALU to the upper immediate rather than performing any calculations. For AUIPC, we added the PC value and the extended immediate. The output of both of these is put into the mux that decides the write data to the register file.

For the S-type instructions, we did something similar to the load instructions. We took the least significant byte or half word and inserted it into the read data from the memory. The location in the read data was determined by an immediate, where 0 maps to the least significant byte and 4 maps to the most significant byte and 0-1 maps to the least significant half word and 2-3 maps to the most significant byte. A mux for byte and half word returns the data with the byte or half word inserted. Then a mux is used to decide between full word, byte, and half word using Func3. This output would then be written to the data memory.

For the J-type instructions, we only needed to implement the JALR instruction. The JAL instruction added an immediate to the PC value for the PCTarget . This would then be put into a mux between the regular PC+4 value and the PCTarget value. A value called PCSrc is used to decide between the two values, which will be talked about later. For JALR, we put the output of read data one and the extension of the immediate into the ALU and added them together. A mux to determine the PCTarget value was added to decide between the output of the added PC value and the immediate and the added RD1 value and the immediate.

For the B-type instructions, we used the ALU to create four flags for the branch conditions. We created a Zero, Negative, Overflow, and Carry flags for this purpose, with the logic for overflow being provided in the lab documents. In the main decoder, we use Func3 in a case statement to decide which branch instruction was called, assigning the BControl value to specific equations from each branch type involving the flags from the

ALU. Figure 2.2 shows the different equations that decide whether the condition of the branch instruction evaluates to be true or not.

BEQ	BControl = Zero
BNE	BControl = ~Zero
BLT	BControl = Negative ^ Overflow
BGE	BControl = ~(Negative ^ Overflow)
BLTU	BControl = Carry
BGEU	BControl = ~Carry

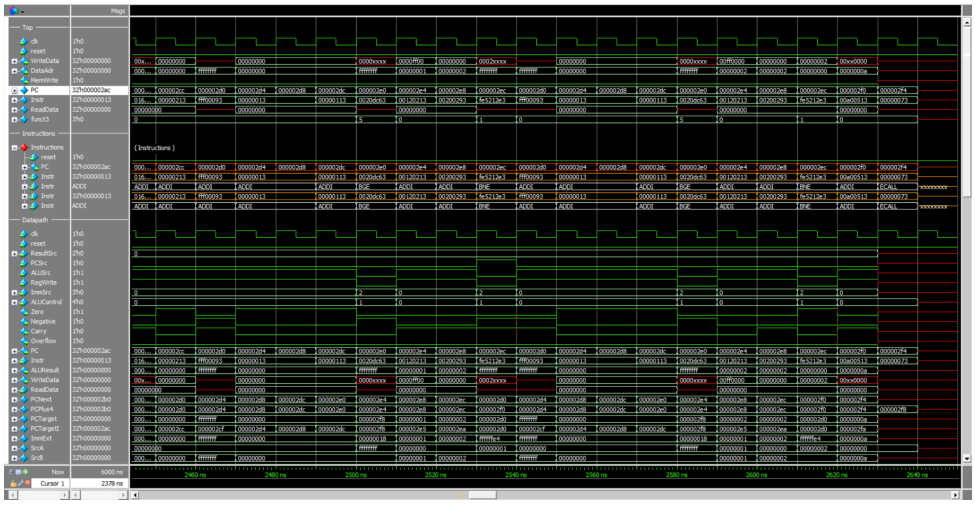
(Figure 2.2 - Branch Instruction Condition Equations)

In order to go to the target PC value, PCSrc is used in the mux between PCTarget and PC+4. The PCSrc is one whenever the instruction is a jump instruction or whenever the instruction is a branch instruction and BControl is one. This ensures that whenever a branch instruction is false, the program moves on to the next instruction and whenever it is true, the program goes to the target value. The target value for the branch instructions is calculated in the same way it is calculated for the JAL jump instruction.

Our design is not fully split into separate modules; however, our design would work modularly if the logic was moved into separate modules. The logic for things like load and store word is not reliant on being in the data path file. The muxes can be moved to separate modules if needed, however we find it easier to understand whenever it is in the datapath module.

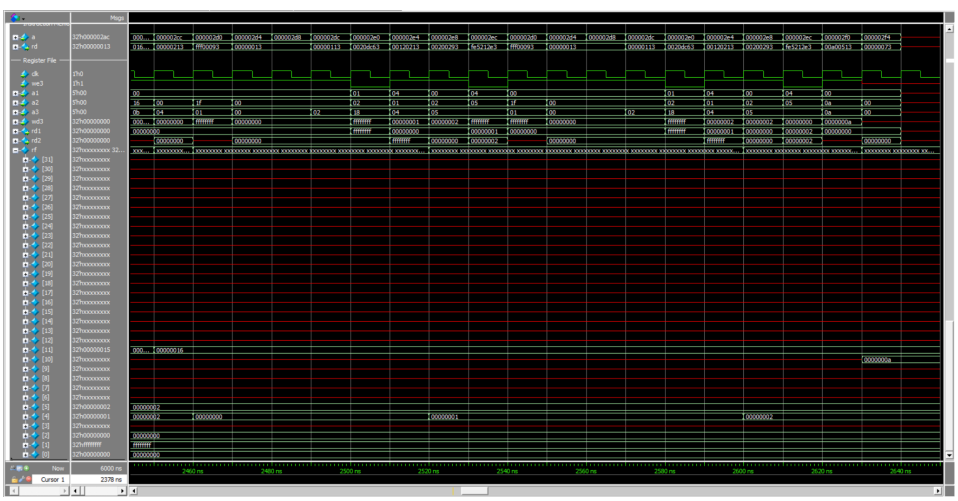
III. Testing Strategy & Simulation

Once the design of an instruction was completed, we tested the instruction with ModelSim. Provided in the lab documents were Assembly and machine code files for validation of each of the instructions required for our design. SystemVerilog code for the testbench was provided, later being modified to perform the tests we wanted. We ran a simulation after completing each instruction, or set of similar instructions, for debugging. Figure 3.1 shows an example of the instruction specific simulations for the BGE instruction.



(Figure 3.1 - Waveform for BGE Instruction)

Whenever a simulation for a specific instruction completed successfully, the next instruction would store a value of 10 into register A0 in the register file to indicate its success. If for whatever reason the instruction did not complete successfully, the next instruction would store a value of 17 into register A0, indicating the instructions failure. Figure 3.2 shows an example of a successfully completed simulation for the BGE instruction.

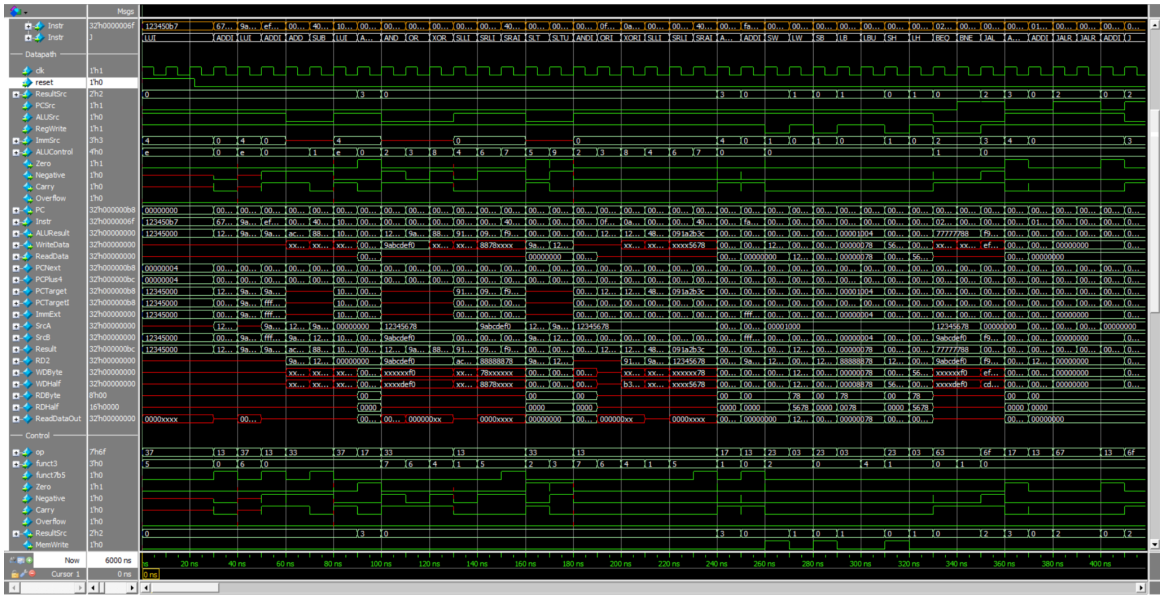


(Figure 3.2 - Successful Simulation for BGE Instruction)

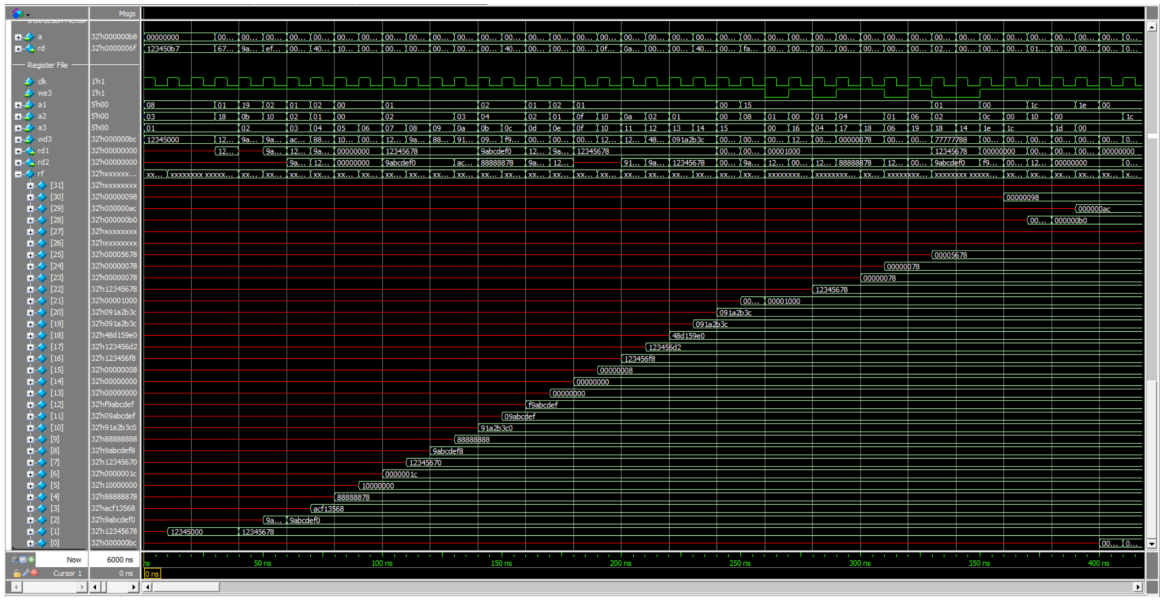
All instruction simulations passed, with the exception of load and store instructions. This was due to the program design not accounting for the lack of data in the data memory. We attempted to rectify this issue by instantiating all the data memory to begin with the values designated at the beginning of the Assembly files, which resulted in the simulations succeeding. We also created our own validation test to validate all load

and store instructions. We also verified that the instructions were working properly with the data memory given in the simulations that were failing because of the unclear data memory.

After validating all the instructions with their respective simulations, we ran both the test_hw and test_hw2 simulations as specified in the lab instructions. Both simulations were successful, with each instruction being verified manually. Figure 3.3 and Figure 3.4 show the datapath and register file waveforms of the test_hw simulation.



(Figure 3.3 - Datapath Waveform of test_hw Simulation File)



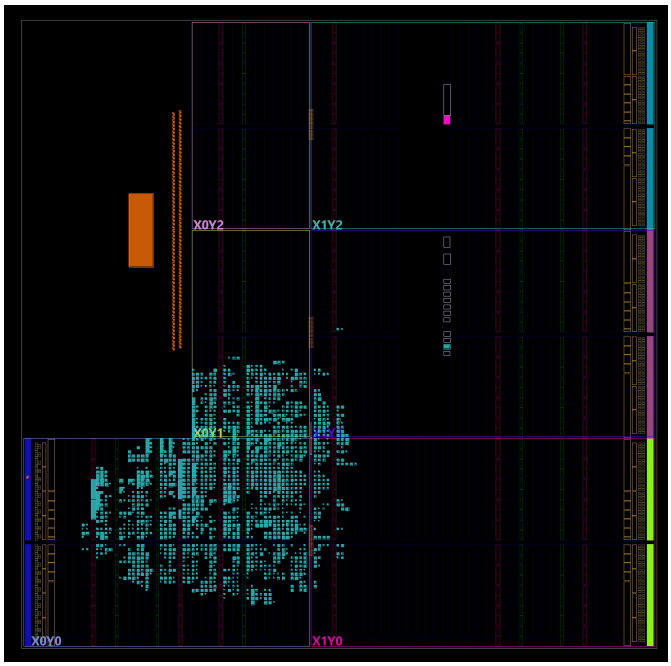
(Figure 3.4 - Register FIle Waveform of test_hw Simulation File)

IV. Implementation

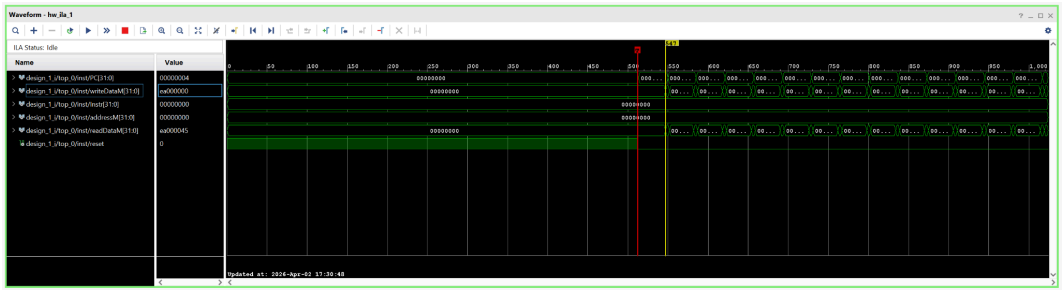
After all instructions had been validated, we implemented the design onto a FPGA/DSDB board using Vivado. In Vivado, we had to change parts in order to synthesize our design correctly. The major change was removing the testbench and top modules as they were not needed for actual implementation of the processor design. We also removed the Imem and Dmem modules from the files since different ones were provided to be aligned with the FPGA board.

Another major change was adjusting our design to include Memstrobe and PCReady. Memstrobe is an output of the main module, informing the connecting architecture that the current instruction is a load or a store instruction to allow for stalling that keeps the cycle aligned. PCReady is an input to the main module that signals that other modules are ready for the next PC value. These changes required some alterations to the inputs and outputs of modules.

Once the major changes were complete, we made sure that the riscv_single.sv file was in the correct location and the memfile.dat was added to the projects. The memfile.dat file contains the instructions for the test_sw test in machine code, as required by the lab document. We then generated a block diagram, synthesized the design, implemented the design, and programmed the FPGA board with the design. We made sure the board was set to QSPI and that we pressed BTN0 at the start of the run to reset it. Figure 4.1 shows the implementation on the board and Figure 4.2 shows the waveform of our run.



(Figure 4.1 - Design Implementation on FPGS/DSDB Board)



(Figure 4.2 - Waveform of Vivado Run)

Our design synthesized successfully and was implemented on to the board. The waveform changes to different values, indicating that the design is working properly on the FPGA/DSDB board. Shown in the images are some of the values the design is reading and writing to the memory.

V. Conclusion

In this lab, we utilized our knowledge from previous labs and lectures to build a RISC-V 32I single cycle processor. We designed the datapath of missing instructions in SystemVerilog, simulated and validated all instructions through assembly programs using ModelSim, and implemented the design onto a FPGA/DSDB board using Vivado. The lab exemplified the general design process of a processor, which is an important part of the computer engineering field. Designing a RISC-V 32 bit instruction set gives us valuable experience in instruction set architecture, using a processor that is being utilized more for real world applications.

VI. References

[1] Shahriar Shahabuddin, Lab 2: Single-Cycle RISC-V Microarchitecture Implementation in SystemVerilog HDL, Oklahoma State University, Stillwater, OK, USA, 2026.